

Assignment/Quiz-RNN/LSTM/GRU**1. Modify the provided notebooks for RNN, LSTM, and GRU, Add another layer of the same type to each model:**

- Add another **RNN layer** to the RNN model.
- Add another **LSTM layer** to the LSTM model.
- Add another **GRU layer** to the GRU model.

Typology	N1 SimpleRNN	N2 LSTM	N3 GRU
One to one	cell 15	cell 15	cell 15
Many to one	cell 19	cell 19	cell 19
One to many	cell 23	cell 23	cell 23
Synchronized many to many	cell 27	cell 27	cell 27
Sequence to sequence	cell 31	cell 31	cell 31

2. Obtain all performance metrics for each typology of each model (RNN, LSTM, and GRU):

The notebooks analyze four metrics of regression: MAE, MSE, RMSE, and R-squared (R^2). The lower values of MAE, MSE, and RMSE mean less prediction error, while the higher value of R^2 shows that more variability is explained by the model.

A. Stacked SimpleRNN

Typology	MAE	MSE	RMSE	R^2
One-to-One	2.3779	10.1762	3.1900	0.8371
Many-to-One	1.6305	4.5548	2.1342	0.9273
One-to-Many	4.6939	40.5350	6.3667	0.3517
Many-to-Many Synced	1.6512	4.9548	2.2259	0.9209
Many-to-Many Seq2Seq	3.4140	18.9464	4.3527	0.6978

In the case of SimpleRNNs arranged in stacks, the Many-to-One structure yields the minimum error rate out of all five structures for SimpleRNNs (RMSE = 2.1342) and an R^2 value of 0.9273. The worst performance is obtained by the One-to-Many structure (RMSE = 6.3667, R^2 = 0.3517).

B. Stacked LSTM

Typology	MAE	MSE	RMSE	R^2
One-to-One	2.0164	8.3284	2.8859	0.8667
Many-to-One	1.5223	4.1927	2.0476	0.9331
One-to-Many	4.7488	43.6455	6.6065	0.3019
Many-to-Many Synced	1.6367	5.0560	2.2486	0.9193
Many-to-Many Seq2Seq	3.4794	20.8412	4.5652	0.6675

In LSTM, the Many-to-One approach is the best also. Its value of MAE is 1.5223, RMSE is 2.0476, and R^2 is 0.9331. In the same way, the One-to-Many architecture is the hardest to work with, giving the biggest error and smallest R^2 .

C. Stacked GRU

Typology	MAE	MSE	RMSE	R ²
One-to-One	2.1522	9.0385	3.0064	0.8553
Many-to-One	1.5252	4.0946	2.0235	0.9346
One-to-Many	4.7884	43.0182	6.5588	0.3120
Many-to-Many Synced	1.5875	4.7298	2.1748	0.9245
Many-to-Many Seq2Seq	3.0439	16.1553	4.0194	0.7423

Again, for GRU, the Many-to-One model gives the best results, with MAE = 1.5252, RMSE = 2.0235, and $R^2 = 0.9346$. The RMSE obtained by this model is just marginally lower than that of the LSTM model.

3. Determine which typology performs best after adding another layer of the corresponding type (RNN, LSTM, or GRU). Discuss the results based on the performance metrics.

Model	Best typology	MAE	RMSE	R ²
Stacked SimpleRNN	Many-to-One	1.6305	2.1342	0.9273
Stacked LSTM	Many-to-One	1.5223	2.0476	0.9331
Stacked GRU	Many-to-One	1.5252	2.0235	0.9346

In terms of all three types of recurrent family structures, the Many-to-One architecture proves to be the most efficient one. It is suitable for energy forecasting due to the fact that this type of network uses a certain number of observations and gives one output at the end.

Of all three notebook results, the stacked GRU provides the minimum Many-to-One RMSE value (2.0235) as well as the maximum R^2 (0.9346). Stacked LSTM is very close to it, whereas SimpleRNN has marginally higher error values. It can be observed that the gated units in LSTM and GRU help in modeling the dependency relationships in the input sequences. In this case, GRU performs better despite having a simpler gating mechanism than LSTM.

Comparing the various typologies still needs to be done cautiously since they do not predict the same type of output structure. This is because One-to-One predicts one value from one step while Seq2Seq forecasts a sequence of future values. Thus, the most appropriate comparison is when they have the same typology and forecast target.

4. Advanced Challenge: Add one CNN layer before the RNN layer and run the model for **100 epochs** using the following settings:

- Number of filters: **32**
- Filter/kernel size: **3**

The advanced challenge adds a one-dimensional convolution before the RNN. The notebook uses 32 filters, kernel size 3, ReLU activation, and 100 training epochs, as required.

```

: tf.keras.backend.clear_session()
tf.random.set_seed(42)

cnn_rnn_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(WINDOW, N_FEATURES)),
    tf.keras.layers.Conv1D(filters=32, kernel_size=3, padding="same", activation="relu"),
    tf.keras.layers.SimpleRNN(UNITS, return_sequences=True),
    tf.keras.layers.SimpleRNN(UNITS, return_sequences=False),
    tf.keras.layers.Dense(1),
], name="cnn_stacked_rnn_many_to_one")

cnn_rnn_model.summary()
show_model_diagram(cnn_rnn_model, "cnn_stacked_rnn_many_to_one.png")

cnn_history = compile_and_fit(
    cnn_rnn_model, "CNN_SimpleRNN_many_to_one",
    x_train, y_train, x_val, y_val, epochs=100
)
plot_history(cnn_history, "CNN + stacked SimpleRNN: Many-to-One")

cnn_pred = cnn_rnn_model.predict(x_test, verbose=0)
cnn_metrics = regression_metrics(y_test, cnn_pred)
pd.DataFrame([{"model": "CNN + stacked SimpleRNN", "typology": "many_to_one",
               "parameters": cnn_rnn_model.count_params(), **cnn_metrics}]).round(4)

```

Model	Typology	Parameters	MAE	MSE	RMSE	R ²
CNN + stacked SimpleRNN	Many-to-One	4,705	1.5861	4.4424	2.1077	0.9291

5. Provide a conclusion regarding the performance of the model using the settings in Item 4.

The CNN-RNN model performed well with MAE = 1.5861, RMSE = 2.1077, and $R^2 = 0.9291$ on Many-to-One architecture. R^2 shows that the model is capable of explaining a high percentage of the variation in the values of the target variable. The Conv1D layer is able to identify short local patterns in neighboring time steps prior to feeding the data to the recurrent network.

Unfortunately, the CNN layer did not automatically result in the best Many-to-One outcome among all possible models. In terms of the comparison provided in the notebook, the best result for recurrent only model is GRU Many-to-One architecture with RMSE = 1.9591 for this training session, while CNN + SimpleRNN resulted in RMSE = 2.1077. According to the notebook, CNN-RNN RMSE is about 7.58% higher than that of the best recurrent-only model. Hence, the convolution helped to improve feature extraction capabilities of the model but was not effective enough to surpass the gated recurrent network for this particular Many-to-One prediction task.

6. Compare the performance of the model in Item 4 with the one-layer RNN model for each typology. Discuss and explain your findings.

The notebook does a direct comparison of 100 epochs on a one-layer SimpleRNN and CNN plus a one-layer SimpleRNN for all five typologies. This satisfies the direct comparison that is needed between Item 4 and the one-layer RNN model.

Typology	Model	MAE	MSE	RMSE	R ²
One-to-One	RNN	2.1223	8.7531	2.9586	0.8599
One-to-One	CNN+RNN	2.2946	10.1119	3.1799	0.8381
Many-to-One	RNN	1.5706	4.3058	2.0750	0.9313

Many-to-One	CNN+RNN	1.6280	4.6229	2.1501	0.9262
One-to-Many	RNN	4.5204	38.4348	6.1996	0.3853
One-to-Many	CNN+RNN	4.4844	37.8274	6.1504	0.3950
Many-to-Many Synced	RNN	1.5942	4.7521	2.1799	0.9242
Many-to-Many Synced	CNN+RNN	1.6102	4.7448	2.1783	0.9243
Many-to-Many Seq2Seq	RNN	4.1104	24.5375	4.9535	0.6086
Many-to-Many Seq2Seq	CNN+RNN	2.9134	14.1174	3.7573	0.7748

Typology	RMSE change with CNN	Better model
One-to-One	+7.48%	One-layer RNN
Many-to-One	+3.62%	One-layer RNN
One-to-Many	-0.79%	CNN + RNN
Many-to-Many Synced	-0.08%	CNN + RNN (very small gain)
Many-to-Many Seq2Seq	-24.15%	CNN + RNN

CNN shows reduction in RMSE value in three out of five matched architectures: One-to-Many, Many-to-Many Synced, and Many-to-Many Seq2Seq. The decrease in RMSE value is negligible for One-to-Many and Many-to-Many Synced, while it is significant for Many-to-Many Seq2Seq, as RMSE falls from 4.9535 to 3.7573, that is, by 24.15%. In case of Many-to-Many Seq2Seq, there is also an increase in R^2 value, which goes up from 0.6086 to 0.7748.

On the contrary, CNN + RNN works poorly for One-to-One and Many-to-One problems. This indicates that the use of convolution is most helpful when the problem to be solved involves generation of an output sequence by learning temporal patterns from local inputs before producing the output sequence. In simple cases where only one output needs to be predicted, the basic RNN is already a good choice.